Find the Promos: a promo shortlister for microdrama screenplays

Problem 1 · App: kukuku-promo-finder.streamlit.app · Code + run steps: github.com/raghuk23/kuku-promo-finder · Raw outputs: `outputs/gemini/` in the repo

1. The problem I picked, and the assumptions I made

Why Problem 1. My background is classic ML, and this problem has a shape I know: generate many candidates cheaply, then rank them well, and ranking is the part an LLM can do and rules can't. It has a real user (an editor scrubbing every episode by hand, weekly) and a checkable output, so I could measure quality instead of arguing about it.

Assumptions (the brief is silent on these):

- **"The best promos" means the best for the whole show, not a quota per episode.** I rank all 50 episodes together. A quiet episode can return nothing: forcing a pick from it would manufacture bad promos. Editors also work episode by episode, so I output a *best per episode* list too, with clips below the bar marked as such.
- **A promo must meet all three criteria** in the brief, so each one is a pass/fail floor (out of 10: hook ≥ 7, cliffhanger ≥ 7, makes sense on its own ≥ 6). I set it from the rubric before seeing any results.
- 20–60 s is a hard limit. A clip never starts or ends mid-line. Timestamps are episode-relative, as in the screenplays.
- The user is an editor choosing what to post. So the output is a shortlist with reasons, not one answer.
- **Not built:** promos stitched from several moments. The output format allows several segments, but every promo today is one continuous stretch.

2. How it works

> **Code owns the clock; the LLM owns the taste.** LLMs are unreliable with numbers like `01:27.21`, so the model never writes a timestamp. It refers to numbered script lines; code converts them to times and checks them.

Step	Who	What it does
1. Parse	Code	50 screenplays → 3,727 timed lines (dialogue, action, camera), with 0 anomalies. It tolerates inconsistent formatting, such as label case, stacked tags and name variants.
2. Candidates	Code + rules	Lists every 20–60 s window that starts and ends on a camera cut, never inside a line: 12,127 windows. A keyword/structure score rewards conflict, physical action and cliff markers in the first 3 s and last 4 s. After dropping overlapping windows, the top 3–8 per episode (by length) are kept: **250 candidates**. This step is cheap and deterministic, and it caps the LLM spend.
3. Judge	LLM (Gemini 3.8 Flash)	Reads each candidate's lines with 8 s of context and scores it against a rubric with anchored examples. It returns the hook, the cliffhanger, why the clip works, scores 1–10 for hook, cliffhanger and making sense on its own, a weakness, and optionally a tighter cut (by line number). The show's summary is withheld, so "on its own" is judged cold. Settings: temperature 0, JSON schema output, 5 episodes per call. Each batch is judged in **two candidate orders and the scores averaged** (see §4).
4. Validate	Code	Every cut must land on a real cut and be 20–60 s long. Every quoted line must exist inside the clip. Bad trims are rejected, or reverted to the original cut.
5. Rank	Code	Score = 0.45 hook + 0.30 cliffhanger + 0.25 on-its-own. The hook gets the most weight because on a feed, a clip nobody stops for never reaches its cliffhanger. Then: apply the floor, remove duplicates, allow at most 2 per episode, and add a small bonus for early episodes (a new viewer lands near the start of the show). The bonus fades to 0 by episode 20.

Result on the input pack: 12,127 windows → 250 candidates → **58 pass the floor**, in 37 of 50 episodes → top 10 in posting order. Top 3: ep 31 02:10.80–02:40.00 (branding-iron threat, 9.3/10); ep 24 00:28.80–01:10.50 (riding-crop lash, 9.2); ep 16 00:16.80–00:44.20 (street fight ending on "Who sent you?", 8.9). The full list, with hook, cliffhanger and reason, is in `outputs/gemini/promos_ranked.md`. There are two ways to run it: a CLI, and a Streamlit app that shows these results or runs on newly uploaded screenplays with a pasted API key.

3. Scale and cost

- **Per show (50 episodes):** 250 candidates → 10 batched calls × 2 orders = **20 LLM calls**. Measured over 8 live calls: about 37k input and 9k output+thinking tokens per call. That makes about **0.73M input and 0.18M output tokens per show**. For example, at $0.50/M input and $3/M output, that is about $0.90 per show.
- **Free tier:** Gemini allows 20 requests per model per day, which is exactly one show a day. That is why episodes are batched 5 per call. Every answer is also cached by its content, so re-runs, ranking changes and resumed runs cost nothing.
- **Time:** about 30–40 min for 50 episodes on the free tier, with calls run one after another (2–8 min each). Batches are independent, so on a paid tier they run in parallel and a show takes minutes.
- **Swapping the model** takes one config line, CLI flag or app setting. Any OpenAI-compatible endpoint works. Each run uses exactly one model: cached answers from a different model are re-judged, so a ranking never mixes judges.
- **The human stays in the loop:** the editor picks from the top 10 and the per-episode list. The app flags borderline clips and quote mismatches for a closer look.

4. How I know it's good, and where it fails

Check	Result	What it tells me
Timestamp validity (250 clips)	100% land on real cuts, 20–60 s	Guaranteed by code, not by trusting the model.
Same prompt run twice (54 clips)	100% identical scores	The judge is repeatable at temperature 0.
Same clips in shuffled order (54 clips)	Rank correlation 0.64; **26% of post/don't-post decisions flipped**	The judge has a real order bias. Fix: average two orders.
After the fix: 1 order vs average of 2 (250 clips)	Rank correlation 0.95; 92% same decision; 8 of 10 top picks unchanged. 45 clips (18%) still get different verdicts in the two orders.	Those 45 sit a median 0.5 points from the floor, and none are in the top 20. The app labels them "borderline".
Ablation: rules only vs LLM (250 clips)	Rank correlation 0.29; 2 of 10 top picks shared. The rules' top 10 are all tied at 10.0; the LLM's spread from 8.32 to 9.30.	Rules can shortlist but can't rank. The LLM is what makes ranking possible.
Blind second-model check (25 clips: the single-order run's top 10, its 5 weakest and 10 random). A fresh Claude agent that had seen no scores rated each clip 1–5. *This is not a human review.*	Rank correlation with Claude: Gemini (2 orders) **0.82**; rules only **0.40**. Of the clips Gemini would post, Claude rated 82% good (4+). Of the clips Claude rated good, Gemini would post 75%.	Two different model families agree, and both disagree with the rules. But LLMs may share taste biases, so a human review is still the missing check. (eval/claude_agreement.md)

Where it picks badly

- **The LLM's trims are unreliable.** Of 124 tighter cuts it suggested, 63 (51%) were unusable: 45 referred to line numbers it wasn't given, 17 broke the length or cut rules, and 1 cut off its own cliffhanger (reverted). Code catches all of them, but trimming should stay an editor's job.
- **Order bias near the bar.** A borderline clip's verdict depends on which clips it is shown next to. Averaging two orders narrows this but does not remove it.
- **Recall is unmeasured.** The LLM only sees the 250 windows the rules let through (2% of 12,127). A great quiet moment with no conflict words or cut markers never reaches it. 13 episodes got no promo (2, 6, 8, 9, 17, 19, 22, 33, 35, 37, 38, 48, 49), 10 of them for lack of a hook. Some are genuinely quiet stretches; I haven't checked which ones the rules simply missed.
- **Quotes:** 4 of 250 write-ups quote a line that isn't in the clip. These are flagged, not auto-fixed.
- **It reads text, not video.** It can't see acting, pacing or whether a slap lands on screen. It doesn't check content safety either: the #1 pick is a branding-iron threat, which the judge itself flagged as possibly too cruel for some ad platforms.
- **The weights and floor are my judgment.** They are not yet tested by a human panel or against how posted promos actually perform.

How I'd know it works in production: log each posted promo's 3-second hold rate and click-through, compare them with the tool's scores, and re-fit the weights and floor. I would also raise recall (more candidates per episode on a paid tier) and add multi-moment stitching.

5. AI tools I used, and where they got it wrong

Claude Code pair-programmed the pipeline and its 134 tests with me. I made the design calls: a quality floor instead of a quota, code-owned timestamps, and two-order averaging. **Gemini 3.8 Flash** (free tier) is the judge. A separate, blind Claude agent was the second judge in §4.

Where AI got it wrong. Claude's first version treated Gemini's "429 rate limit" errors as temporary and retried them with back-off. On the free tier the cap is 20 requests per model *per day*, so every retry burned quota, and a full run stalled partway. I changed it to recognise the daily-quota error and stop at once, to batch 5 episodes per call (50 calls → 10), and to cache every answer so a stopped run resumes for free. Gemini also makes mistakes: half of its suggested trims were unusable (see above). This is why code checks every number it returns.